

Отчёт - Чекпоинт 2: Core RAG

1. Архитектура пайплайна

Реализован полный end-to-end RAG-пайплайн, включающий:

- **Ingestion** → text conversion → chunking → embeddings → BM25/FAISS
- **Retrieval**: BM25, Dense, Hybrid+DAT
- **Генерация ответа** моделью Mistral (mistral-small-latest)
- **Три режима**:
 - Local Dense RAG
 - Hybrid RAG с Dynamic Alpha Tuning
 - Web RAG (Tavily + FAISS reranking)

Архитектура описана в README.

2. Основная логика реализована

Выполнено:

- построен локальный корпус чанков;
- создан BM25 индекс;
- создан FAISS индекс (dense);
- реализован embedder для запросов;
- реализован гибридный retriever с DAT (эвристический α , возможность LLM-scoring);
- реализован пайплайн ScienceRAGPipeline для local / hybrid / web режимов;
- интегрирована модель Mistral для генерации ответов.

Система принимает текстовый запрос и возвращает финальный ответ + supporting context.

3. Планировщик запросов (Query Planner) и итеративный цикл

3.1 Архитектура планировщика

Реализован модуль QueryPlanner (src/pipeline/planner.py), который обеспечивает автоматическое улучшение качества ответов через итеративный цикл уточнения запросов.

Основные компоненты:

1. Оценка качества (Quality Assessment)

- Метод `evaluate_quality()` анализирует качество найденных документов и сгенерированного ответа
- Использует LLM (Mistral) для оценки релевантности контекстов
- Рассчитывает статистику: средний, максимальный и минимальный similarity scores
- Возвращает стратегию улучшения: "sufficient", "reformulate", "increase_top_k", "both"

2. Переформулировка запросов (Query Reformulation)

- Метод `reformulate_query()` переформулирует исходный запрос на основе:
 - Оригинального запроса пользователя
 - Предыдущего ответа (который может быть неполным)
 - Метрик качества retrieval (mean_score, max_score)
- Сохраняет язык оригинального запроса
- Использует LLM для генерации улучшенной формулировки

3.2 Итеративный цикл уточнения

Все три режима RAG (`answer_local`, `answer_local_hybrid`, `answer_web`) реализуют итеративный цикл с параметром `max_planner_iterations` (по умолчанию 2 итерации).

Алгоритм работы:

На каждой итерации система выполняет retrieval, генерирует ответ и оценивает его качество. LLM-модели предлагается выбрать одно из действий:

- **"sufficient"** — качество ответа достаточное, можно завершить цикл
- **"reformulate"** — переформулировать запрос (неправильно подобранные документы)
- **"increase_top_k"** — увеличить количество извлекаемых документов на 3 (недостаточно контекста)
- **"both"** — применить обе стратегии одновременно (переформулировать запрос и увеличить `top_k`)

Цикл продолжается до достижения достаточного качества или максимума итераций. Все переформулировки запросов и метрики сохраняются в `RAGAnswer.refinement_history`.

3.3 Интеграция в пайплайн

Планировщик интегрирован во все три режима работы:

- **Local RAG** (`answer_local`): Использует dense retrieval с итеративным уточнением
- **Hybrid RAG** (`answer_local_hybrid`): Использует гибридный retrieval (BM25 + Dense) с DAT и планировщиком
- **Web RAG** (`answer_web`): Использует Tavily search с последующим FAISS reranking и планировщиком

3.4 Структура данных

Реализованы dataclass'ы для отслеживания процесса уточнения:

- `RefinementInfo`: Информация об одной итерации уточнения (`iteration`, `query`, `top_k`, `mean_score`, `was_refined`, `strategy`)
- `QualityAssessment`: Результат оценки качества (`is_sufficient`, `strategy`, `reasoning`)
- `RAGAnswer`: Финальный ответ с полной историей (`query_history`, `refinement_history`)

4. Подготовлена валидационная выборка

Скрипт `generate_qa_dataset.py` сформировал датасет:

- **859 вопросов**
- Каждый вопрос привязан к "gold" чанк-ID
- Хранится в `data/validation/qa.jsonl`

5. Проведено первичное тестирование

Тестирование всех трёх retrievers выполнено с помощью `evaluate_retrievers.py`

Вычислены метрики R@k и P@k для k=1,3,5.

Результаты (важное кратко)

Retriever	R@1	R@3	R@5
BM25	0.356	0.471	0.508
Dense	0.161	0.263	0.312

Retriever	R@1	R@3	R@5
Hybrid + DAT	0.232	0.346	0.455

Вывод: BM25 остаётся сильным baseline. Hybrid+DAT работает лучше dense, но пока не превосходит BM25 → зона улучшений для последующих итераций (LLM-based scoring α).

6. End-to-end пайплайн

Реализован полный пайплайн от запроса до ответа:

1. **Вход:** Пользовательский запрос через Streamlit UI
2. **Retrieval:** Поиск релевантных документов (local/hybrid/web)
3. **Оценка качества:** Анализ релевантности через QueryPlanner
4. **Итеративное уточнение:** Переформулировка запроса или увеличение контекста при необходимости
5. **Генерация:** Формирование финального ответа моделью Mistral
6. **Выход:** Ответ с источниками и историей уточнений

Пайплайн работает end-to-end через Streamlit приложение (app/streamlit_app.py).

7. Итог

На момент чекпоинта 2:

- ☐ RAG-ядро полностью собрано,
- ☐ Retrieval протестирован,
- ☐ Метрики зафиксированы,
- ☐ Базовая версия работает end-to-end,
- ☐ Реализован планировщик запросов с итеративным циклом уточнения,
- ☐ Добавлена нелинейная логика улучшения качества ответов.